

Lab06 – Multi-Container Pods

Init Container

Kubernetes runs an init container before the main container. In this lab, the init container retrieves configuration files from a remote location and makes it available to the application running in the main container. The configuration files are shared through a volume mounted by both containers. The running application consumes the configuration files and can render its values.

1. Create a new Pod in a YAML file named `business-app.yaml`. The Pod should define two containers, one init container and one main application container. Name the init container `configurer` and the main container `web`. The init container uses the image `busybox`, the main container uses the image `brahimhamdi/nodejs-read-config:1.0.0`. Expose the main container on port 8080.

```
vagrant@kubernetes-control-plane:~/lab$ vim app.yaml
vagrant@kubernetes-control-plane:~/lab$ cat app.yaml
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  name: business-app
spec:
  initContainers:
  - name: configurer
    image: busybox
  containers:
  - image: brahimhamdi/nodejs-read-config:1.0.0
    name: web
    ports:
    - containerPort: 8080

vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

2. Edit the YAML file by adding a new volume of type `emptyDir` that is mounted at `/usr/shared/app` for both containers.

```

vagrant@kubernetes-control-plane:~/lab$ vim app.yaml
vagrant@kubernetes-control-plane:~/lab$ cat app.yaml
apiVersion: v1
kind: Pod
metadata:
  name: business-app
spec:
  initContainers:
  - name: configurer
    image: busybox
    volumeMounts:
    - name: configdir
      mountPath: "/usr/shared/app"
  containers:
  - image: brahimhamdi/nodejs-read-config:1.0.0
    name: web
    ports:
    - containerPort: 8080
    volumeMounts:
    - name: configdir
      mountPath: "/usr/shared/app"
  volumes:
  - name: configdir
    emptyDir: {}

vagrant@kubernetes-control-plane:~/lab$

```

3. Edit the YAML file by providing the command for the init container. The init container should run a `wget` command for downloading the file ``https://raw.githubusercontent.com/brahimhamdi/ckad-lab/master/labs/lab6/config.json`` into the directory ``/usr/shared/app``.

```

vagrant@kubernetes-control-plane:~/lab$ vim app.yaml
vagrant@kubernetes-control-plane:~/lab$ cat app.yaml
apiVersion: v1
kind: Pod
metadata:
  name: business-app
spec:
  initContainers:
  - name: configurer
    image: busybox
    command:
    - wget
    - "-O"
    - "/usr/shared/app/config.json"
    - "https://raw.githubusercontent.com/brahimhamdi/ckad-lab/master/labs/lab6/config.json"
    volumeMounts:
    - name: configdir
      mountPath: "/usr/shared/app"
  containers:
  - image: brahimhamdi/nodejs-read-config:1.0.0
    name: web
    ports:
    - containerPort: 8080
    volumeMounts:
    - name: configdir
      mountPath: "/usr/shared/app"
  volumes:
  - name: configdir
    emptyDir: {}

vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$

```

4. Create the Pod from the YAML file. Once the application is running, shell into the container. The mounted volume path contains the downloaded configuration file. The `curl` command renders the values from the configuration file.

```

vagrant@k8s-control-plane:~/lab$ kubectl apply -f app.yaml
pod/business-app created
vagrant@k8s-control-plane:~/lab$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
business-app  0/1     PodInitializing  0          18s
vagrant@k8s-control-plane:~/lab$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
business-app  0/1     PodInitializing  0          50s
vagrant@k8s-control-plane:~/lab$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
business-app  1/1     Running    0          5m16s
vagrant@k8s-control-plane:~/lab$ kubectl exec business-app -it -- /bin/sh
Defaulted container "web" out of: web, configurator (init)
# ls /usr/shared/app
config.json
# curl localhost:8080
Database URL: localhost:5432/customers
# exit
vagrant@k8s-control-plane:~/lab$

```

Adapter pattern

The adapter pattern helps with providing a simplified, homogenized view of an application running within a container. For example, we could stand up another container that unifies the log output of the application container. As a result, other monitoring tools can rely on a standardized view of the log output without having to transform it into an expected format.

5. Create a new Pod in a YAML file named `adapter.yaml`. The Pod declares two containers. The container `app` uses the image `busybox` and runs the command `while true; do echo "\$(date) | \$(du -sh ~)" >> /var/logs/diskspace.txt; sleep 5; done;`. The adapter container `transformer` uses the image `busybox` and runs the command `sleep 20; while true; do while read LINE; do echo "\$LINE" | cut -f2 -d"|"; >> \$(date +%Y-%m-%d-%H-%M-%S)-transformed.txt; done < /var/logs/diskspace.txt; sleep 20; done;` to strip the log output off the date for later consumption by a monitoring tool. Be aware that the logic does not handle corner cases (e.g. automatically deleting old entries) and would look different in production systems.

```

vagrant@k8s-control-plane:~/lab$ vim adapter.yaml
vagrant@k8s-control-plane:~/lab$ cat adapter.yaml
apiVersion: v1
kind: Pod
metadata:
  name: adapter
spec:
  containers:
    - args:
      - /bin/sh
      - -c
      - 'while true; do echo "$(date) | $(du -sh ~)" >> /var/logs/diskspace.txt; sleep 5; done;'
      image: busybox
      name: app
    - image: busybox
      name: transformer
      args:
      - /bin/sh
      - -c
      - 'sleep 20; while true; do while read LINE; do echo "$LINE" | cut -f2 -d"|"; >> $(date +%Y-%m-%d-%H-%M-%S)-transformed.txt; done < /var/logs/diskspace.txt; sleep 20; done;'
vagrant@k8s-control-plane:~/lab$
vagrant@k8s-control-plane:~/lab$

```

6. Before creating the Pod, define an `emptyDir` volume. Mount the volume in both containers with the path `/var/logs`.

```
vagrant@kubernetes-control-plane:~/lab$ vim adapter.yaml
vagrant@kubernetes-control-plane:~/lab$ cat adapter.yaml
apiVersion: v1
kind: Pod
metadata:
  name: adapter
spec:
  volumes:
    - name: config-volume
      emptyDir: {}
  containers:
    - args:
      - /bin/sh
      - -c
      - 'while true; do echo "$(date) | $(du -sh ~)" >> /var/logs/diskspace.txt; sleep 5; done;'
      image: busybox
      name: app
      volumeMounts:
        - name: config-volume
          mountPath: /var/logs
    - image: busybox
      name: transformer
      args:
      - /bin/sh
      - -c
      - 'sleep 20; while true; do while read LINE; do echo "$LINE" | cut -f2 -d"| " >> $(date +%Y-%m-%d-%H-%M-%S)-transformed.txt; done < /var/logs/diskspace.txt; sleep 20; done;'
      volumeMounts:
        - name: config-volume
          mountPath: /var/logs
vagrant@kubernetes-control-plane:~/lab$
```

7. Create the Pod, log into the container `transformer`. The current directory should continuously write a new file every 20 seconds.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl apply -f adapter.yaml
pod/adapter created
vagrant@kubernetes-control-plane:~/lab$
```

```
vagrant@kubernetes-control-plane:~/lab$ kubectl exec adapter --container=transformer -it -- /bin/sh
/ #
/ # cat /var/logs/diskspace.txt
Sat Aug 12 10:35:45 UTC 2023 | 4.0K /root
Sat Aug 12 10:35:50 UTC 2023 | 4.0K /root
Sat Aug 12 10:35:55 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:00 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:05 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:10 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:15 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:20 UTC 2023 | 4.0K /root
Sat Aug 12 10:36:25 UTC 2023 | 4.0K /root
/ # ls -l
total 48
-rw-r--r-- 1 root root 60 Aug 12 10:36 2023-08-12-10-36-07-transformed.txt
-rw-r--r-- 1 root root 108 Aug 12 10:36 2023-08-12-10-36-27-transformed.txt
drwxr-xr-x 2 root root 12288 Jul 17 18:30 bin
drwxr-xr-x 5 root root 360 Aug 12 10:35 dev
drwxr-xr-x 1 root root 4096 Aug 12 10:35 etc
drwxr-xr-x 2 nobody nobody 4096 Jul 17 18:30 home
drwxr-xr-x 2 root root 4096 Jul 17 18:30 lib
lrwxrwxrwx 1 root root 3 Jul 17 18:30 lib64 -> lib
dr-xr-xr-x 168 root root 0 Aug 12 10:35 proc
drwx----- 1 root root 4096 Aug 12 10:36 root
dr-xr-xr-x 13 root root 0 Aug 12 10:35 sys
drwxrwxrwt 2 root root 4096 Jul 17 18:30 tmp
drwxr-xr-x 4 root root 4096 Jul 17 18:30 usr
drwxr-xr-x 1 root root 4096 Aug 12 10:35 var
/ # cat 2023-08-12-10-36-07-transformed.txt
4.0K /root
4.0K /root
4.0K /root
4.0K /root
4.0K /root
/ # exit
vagrant@kubernetes-control-plane:~/lab$
```

Ambassador Pattern

In this example, you'll be asked to implement rate-limiting functionality for HTTP(S) calls to an external service. For example, the requirements for the rate limiter could say that an application can only make a maximum of five calls every 15 minutes. Instead of strongly coupling the rate-limiting logic to the application code, it will be provided by an ambassador container.

The image `brahimhamdi/nodejs-business-app:1.0.0` represents a Node.js-based application that makes a call to localhost on port 8081. The ambassador container represented by the image `brahimhamdi/nodejs-ambassador:1.0.0` running on port 8081 will take on making the HTTP call to the external service while at the same time enforcing rate limiting.

8. Create a YAML manifest for a Pod with the name `rate-limiter` in the namespace `ext-access`. Store the definition in the file named `rate-limiter.yaml`. The main application container named `business-app` should use the image `brahimhamdi/nodejs-business-app:1.0.0` and expose the container port 8080.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl create ns ext-access
namespace/ext-access created
vagrant@kubernetes-control-plane:~/lab$ vim rate-limiter.yaml
vagrant@kubernetes-control-plane:~/lab$ cat rate-limiter.yaml
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
  namespace: ext-access
spec:
  containers:
  - name: business-app
    image: brahimhamdi/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080

vagrant@kubernetes-control-plane:~/lab$
vagrant@kubernetes-control-plane:~/lab$
```

9. Modify the YAML manifest by adding the ambassador container named `ambassador`. The ambassador container uses the image `brahimhamdi/nodejs-ambassador:1.0.0` and exposes the container port 8081.

```
vagrant@kubernetes-control-plane:~/lab$ vim rate-limiter.yaml
vagrant@kubernetes-control-plane:~/lab$ cat rate-limiter.yaml
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
  namespace: ext-access
spec:
  containers:
  - name: business-app
    image: brahimhamdi/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
  - name: ambassador
    image: brahimhamdi/nodejs-ambassador:1.0.0
    ports:
    - containerPort: 8081
vagrant@kubernetes-control-plane:~/lab$
```

10. Test the rate-limiting functionality. First, create the Pod, shell into the `business-app` container that runs the business application, and execute a series of `curl` commands that target `localhost:8081`, the end point of the `ambassador` container. The first five calls will be allowed to the external service. On the sixth call, you'll receive an error message, as the rate limit has been reached within the given time frame.

```
vagrant@kubernetes-control-plane:~/lab$ kubectl exec rate-limiter -it -c business-app -n ext-access -- /bin/sh
# curl localhost:8080/test
{
  "args": {
    "test": "123"
  },
  "headers": {
    "x-forwarded-proto": "https",
    "x-forwarded-port": "443",
    "host": "postman-echo.com",
    "x-amzn-trace-id": "Root=1-64d76b9d-65d6b44629fb19611d210528"
  },
  "url": "https://postman-echo.com/get?test=123"
}# curl localhost:8080/test
{
  "args": {
    "test": "123"
  },
  "headers": {
    "x-forwarded-proto": "https",
    "x-forwarded-port": "443",
    "host": "postman-echo.com",
    "x-amzn-trace-id": "Root=1-64d76bba-5715d63367808cde64c0b5aa"
  },
  "url": "https://postman-echo.com/get?test=123"
}# curl localhost:8080/test
```

...

```
}# curl localhost:8080/test
{
  "args": {
    "test": "123"
  },
  "headers": {
    "x-forwarded-proto": "https",
    "x-forwarded-port": "443",
    "host": "postman-echo.com",
    "x-amzn-trace-id": "Root=1-64d76bc3-2b31cb5911d9ed606bcd461"
  },
  "url": "https://postman-echo.com/get?test=123"
}# curl localhost:8080/test
Too many requests made created from this IP, please try again after an hour#
# exit
vagrant@kubernetes-control-plane:~/lab$
```